

CSA0501
DATABASE MANAGEMENT SYSTEMS
LAB EXPERIMENT

K.NIKHIL

192424366

EXP 6:

AIM:

To view records from tables using SELECT commands with the BETWEEN operator, IN operator, and aggregate functions (AVG, COUNT, SUM, MIN, MAX).

SELECT with Various Clauses – BETWEEN, IN, Aggregate Functions

ALGORITHM:

1. Connect to csa05_lab.
2. Add CourseType to COURSE and Score to STUDENT_COURSE.
3. Populate course types and enrollment scores.
4. Use IN to list courses of type "Statistics"/"Programming".
5. Use BETWEEN to find scores in the 50–80 range.
6. Use AVG() for a specific course's average mark.
7. Use MAX()/MIN() for a specific course.
8. Use GROUP BY with MAX(), MIN(), AVG() per course.
9. Use SUM(), and COUNT() grouped by course and by student.
10. Execute, verify, and stop.

QUESTIONS:

IN & BETWEEN

1. List the type of the courses “Statistics” and “Programming”.
2. The instructor wants to know the CourseNos whose scores are in the range 50 to 80.

AGGREGATE:

1. Find the average mark of “C002”.
2. List the maximum, minimum mark for “C002”.
3. List the maximum, minimum, average mark for each course.
4. List the name of the courses and average mark of each course.
5. Calculate the sum of all the scores.
6. How many students are registered for each course?

7.Display the course description and the number of students registered in each course.

8.How many courses did each student register for?

OUTPUTS:

IN & BETWEEN:

```
mysql> -- Q1: Courses of type "Statistics" and "Programming"
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM COURSE
      -> WHERE CourseType IN ('Statistics', 'Programming');
+-----+-----+-----+-----+-----+
| CourseNo | CourseName          | SemNo | HallNo | CourseType |
+-----+-----+-----+-----+-----+
|      502 | DAA                  |      4 | H002   | Programming |
|      505 | Probability & Stats |      3 | H002   | Statistics  |
+-----+-----+-----+-----+-----+
2 rows in set (0.02 sec)
```

```
mysql> -- Q2: Enrollments with scores between 50 and 80
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT RegNo, CourseNo, Score
      -> FROM STUDENT_COURSE
      -> WHERE Score BETWEEN 50 AND 80;
+-----+-----+-----+
| RegNo | CourseNo | Score |
+-----+-----+-----+
|    101 |      501 |    75 |
|    103 |      501 |    55 |
|    107 |      501 |    65 |
|    108 |      502 |    72 |
|    101 |      502 |    60 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

AGGREGATE:

```
mysql> -- Q1: Average mark for CourseNo 502 ("C002")
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT AVG(Score) AS AverageMark
-> FROM STUDENT_COURSE
-> WHERE CourseNo = 502;
```

```
+-----+
| AverageMark |
+-----+
|      73.3333 |
+-----+
1 row in set (0.01 sec)
```

```
mysql> -- Q2: Max, min mark for CourseNo 502
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT MAX(Score) AS MaxMark, MIN(Score) AS MinMark
-> FROM STUDENT_COURSE
-> WHERE CourseNo = 502;
```

```
+-----+-----+
| MaxMark | MinMark |
+-----+-----+
|      88 |      60 |
+-----+-----+
1 row in set (0.02 sec)
```

```
mysql> -- Q3: Max, min, avg mark per course + course name
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT C.CourseNo, C.CourseName,
->         MAX(SC.Score) AS MaxMark,
->         MIN(SC.Score) AS MinMark,
->         AVG(SC.Score) AS AvgMark
-> FROM COURSE C
-> JOIN STUDENT_COURSE SC ON C.CourseNo = SC.CourseNo
-> GROUP BY C.CourseNo, C.CourseName;
```

```
+-----+-----+-----+-----+-----+
| CourseNo | CourseName          | MaxMark | MinMark | AvgMark |
+-----+-----+-----+-----+-----+
|      501 | DBMS                 |      75 |      55 | 65.0000 |
|      502 | DAA                  |      88 |      60 | 73.3333 |
|      503 | Operating Systems    |      45 |      45 | 45.0000 |
|      505 | Probability & Stats  |      90 |      90 | 90.0000 |
+-----+-----+-----+-----+-----+
4 rows in set (0.02 sec)
```

```
mysql> -- Q4: Sum of all scores
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT SUM(Score) AS TotalScore FROM STUDENT_COURSE;
+-----+
| TotalScore |
+-----+
|          550 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> -- Q5: Number of students registered per course
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT CourseNo, COUNT(*) AS NumStudents
-> FROM STUDENT_COURSE
-> GROUP BY CourseNo;
```

```
+-----+-----+
| CourseNo | NumStudents |
+-----+-----+
|        501 |          3 |
|        502 |          3 |
|        503 |          1 |
|        505 |          1 |
+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> -- Q6: Course description + number of students registered
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT C.CourseNo, C.CourseName, COUNT(SC.RegNo) AS NumStudents
-> FROM COURSE C
-> JOIN STUDENT_COURSE SC ON C.CourseNo = SC.CourseNo
-> GROUP BY C.CourseNo, C.CourseName;
```

```
+-----+-----+-----+
| CourseNo | CourseName      | NumStudents |
+-----+-----+-----+
|        501 | DBMS            |          3 |
|        502 | DAA             |          3 |
|        503 | Operating Systems |          1 |
|        505 | Probability & Stats |          1 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> -- Q7: Number of courses each student registered for
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT RegNo, COUNT(*) AS NumCourses
       -> FROM STUDENT_COURSE
       -> GROUP BY RegNo;
```

RegNo	NumCourses
101	2
103	1
105	1
106	1
107	1
108	1
109	1

```
7 rows in set (0.00 sec)
```

RESULT:

Records from the tables were successfully displayed using SELECT commands with the BETWEEN operator, IN operator, and aggregate functions (AVG, MAX, MIN, SUM, COUNT), including per-course and per-student statistics grouped using GROUP BY.